

Nguyên lý Triển khai Backend Service

Nhóm 10

Nội dung chính

01

Tổng quan về OpenAPI Specification

02

OpenAPI Generator – Sinh code tự động

03

Tổng quan về MongoDB

04

Kết nối API với MongoDB

OpenAPI Specification là gì?

OpenAPI Specification (OAS) là một định dạng mô tả API dành cho REST APIs, được viết bằng YAML hoặc JSON, dễ đọc, dễ hiểu cho cả người dung lẫn ngôn ngữ máy tính

- ✓ **Format chuẩn**
JSON hoặc YAML, dễ đọc và dễ bảo trì
- ✓ **Định nghĩa đầy đủ**
Endpoints, HTTP methods, parameters, request/response schemas
- ✓ **Authentication**
Hỗ trợ API keys, OAuth2, JWT, Basic Auth

```
1  openapi: 3.0.0
2  info:
3    title: Simplified Library Management API
4    version: 1.0.0
5    description: A clean, direct API for library management designed for Week 7.
6
7  servers:
8    - url: http://localhost:8080/api/v1
9
10 components:
11   securitySchemes:
12     bearerAuth:
13       type: http
14       scheme: bearer
15       bearerFormat: JWT
16
17   schemas:
18     # Model for POST/PUT requests (no ID)
19     BookRequest:
20       type: object
21       required:
22         - title
23       properties:
24         title:
25           type: string
26         author:
27           type: string
28         quantity:
29           type: integer
30           default: 1
31
32     # Model for GET requests (includes ID)
```

Lợi ích của Spec-Driven Development



API First Approach

Thiết kế API trước khi viết code. Đảm bảo tính rõ ràng trong yêu cầu.

Thiết kế → Cài đặt



Single Source of Truth

Một nguồn duy nhất cho cả team: Frontend, Backend, QA, DevOps đều dựa vào.

Tránh trùng lặp và thiếu nhất quán



Tự động hóa

Tự động sinh: Documentation, Code (client/server), Tests, Mock servers

Giảm thời gian cho việc viết boilerplate code



Cải thiện hiệu suất làm việc

Frontend và Backend có thể làm việc song song. Frontend dùng mock server, Backend cài đặt theo spec.

OpenAPI Generator

Sinh code Backend

OpenAPI Generator là công cụ tạo server stubs và client SDKs từ OpenAPI Spec.



Server Stubs

Tạo boilerplate code cho API server: Java Spring Boot, Node.js/Express, Python Flask, Go net/http, ...



Client SDKs

Tạo client libraries: JavaScript, Java, Python, Kotlin, ...

```
npm install @openapitools/openapi-generator-cli
```

```
openapi-generator-cli generate -i demo.yaml -g  
python-flask -o ./out
```

```
▼ out  
  > .openapi-generator  
  > .tox  
  > build  
  > openapi_server  
  > openapi_server.egg-info  
  ≡ .coverage  
  🐳 .dockerignore  
  💎 .gitignore  
  ≡ .openapi-generator-ignore  
  ≡ .python-version  
  ! .travis.yml  
  🐳 Dockerfile  
  💰 git_push.sh  
  ⓘ README.md  
  ≡ requirements.txt  
  🐳 setup.py  
  ≡ test-requirements.txt  
  ≡ tox.ini
```

Quy trình sinh code qua OpenAPI Generator

1

Viết OpenAPI Spec

Tạo file YAML/JSON định nghĩa API



2

Chạy Generator

Chọn target language và generate code



3

Cài đặt logic

Viết logic cho các hàm tương ứng các endpoint

4

MongoDB

MongoDB

Là một hệ quản trị CSDL NoSQL, hướng tài liệu (Document-Oriented). Dữ liệu được lưu trữ dưới dạng BSON khá giống JSON.

Các khái niệm cơ bản

Collection

Tương ứng với Table

Document

Tương ứng với Row

Field

Tương ứng với Column

Có đặc điểm Schema-less nên dễ dàng thay đổi cấu trúc CSDL

Một Document mô tả một quyển sách trong Collection books:

```
{
  "_id": "65f1a2b3c4d5e6f7g8h9i0",
  "title": "The Art of War",
  "author": "Sun Tzu",
  "tags": ["War", "Strategy"],
  "available": true
}
```

MongoEngine

MongoEngine

Là một thư viện ODM (Object-Document Mapper) dành cho Python. Đóng vai trò trung gian ánh xạ các Class trong Python thành các Document trong MongoDB

Các thành phần cốt lõi

Document (Model)

Mỗi Collection được đại diện bởi một Class kế thừa từ Document

```
3 class Book(Document):
4     title = StringField(required=True)
5     author = StringField()
6     quantity = IntField(default=1)
7     meta = {'collection': 'books'}
```

QuerySet (Truy vấn)

Sử dụng hàm .objects() để thao tác với dữ liệu

```
User.objects().order_by('id').limit(limit)
Borrowing.objects(user = user_id).select_related()
AuthAccount.objects(username=data['username']).first()
```

Persistence (Lưu trữ)

Lưu trữ dữ liệu

```
new_book = Book(
    title = data['title'],
    author = data.get('author', "Unknown"),
    year = data.get('year', 2026)
)
new_book.save()
```


Kết nối API với MongoDB

1

Tài thư viện MongoEngine

```
pip install mongoengine
```

2

Kết nối với MongoDB

```
connect(host='mongodb://user:password@localhost:port/database?authSource=admin')
```

3

Tạo các Document

Viết Class kế thừa từ Document:

```
class Book(Document):
    title = StringField(required=True)
    author = StringField(default="Unknown")
    year = IntField(default=2026)
    meta = {
        'collection': 'books',
        'indexes': ['title']
    }
```

4

Truy vấn dữ liệu

Ví dụ: Tìm cuốn sách dựa theo id

Hàm `wrap_with_metadata_v2` sẽ biến thực thể thành một DTO (data transfer object) rồi trả về client.

```
@ns_books.route('/<string:book_id>')
class BookItem(Resource):
    @jwt_required()
    def get(self, book_id):
        """Get a specific book"""
        book = Book.objects.get(id=book_id)

        return wrap_with_metadata_v2(book, book_model), 200
```